

Casting Defect Detection

Building, deploying and governing a deep-learning inspection model

MLOps capstone report. It documents the full lifecycle of an image classifier for cast pump impellers: framing the business problem, validating and versioning the data, transfer learning, experiment tracking, model registration, API serving, containerisation and continuous integration, and the monitoring, drift-detection, retraining and approval controls that keep the model trustworthy after release.

Dataset	Public casting product image set (submersible-pump impellers), 7,348 greyscale images
Model	ResNet18 pretrained on ImageNet, frozen feature extractor, new two-class output layer
Headline result	452 of 453 defective test castings detected (recall 0.998); F1 0.953; ROC-AUC 0.997
Production model	casting_defect_classifier, version 4, holding the @production alias in MLflow
Tooling	PyTorch 2.8, MLflow 3.1.4, FastAPI, Docker, GitHub Actions, PSI and embedding drift
Evidence base	Data_Preparation, Model_Development_and_Tracking and Operations_Monitoring_and_Evidence notebooks

Contents

Executive summary	4
1. Introduction	5
Background	5
Scope and approach	5
2. Business and data understanding	6
1.1.1 Business objective and justification	6
1.1.2 Choice of success metric	6
1.1.3 Dataset structure and classes	7
3. Data quality and dataset versioning	7
1.2.1 – 1.2.3 Automated quality checks	7
1.3.1 Reproducible, stratified split	8
1.3.2 Snapshot and metadata	8
4. Exploratory analysis and workflow design	9
1.4.1 What the images look like	9
1.4.2 End-to-end MLOps workflow	10
5. Preprocessing, model, training and tracking	12
2.1.1 Preprocessing for the pretrained network	12
2.1.2 Augmentation for training only	12
2.2.1 – 2.2.2 Transfer-learning architecture	13
2.3.1 Loss, optimiser and seeding	13
2.3.2 Epoch loop, validation and early stopping	13
2.4.1 – 2.4.2 Experiment tracking with MLflow	14
6. Evaluation, registry and deployment	15
3.1.1 Test-set metrics and model selection	15
3.1.2 Failure analysis	15
3.2.1 – 3.2.2 Registration, promotion and version history	16
3.3.1 Inference service	17
3.3.2 Live demonstration	17
3.4.1 Container image	18
3.4.2 Continuous integration	18
7. Monitoring and drift detection	20
4.1.1 Prediction logging	20
4.2.1 – 4.2.2 Feature drift measured with PSI	20
4.3.1 Embedding extraction	22
4.3.2 Drift in feature space	22

8. Retraining and release governance	23
4.4.1 Measurable triggers	23
4.4.2 Drift-aware candidate model	23
4.5.1 Champion versus challenger	23
4.5.2 Promotion rule and decision	23
9. Risk, reproducibility and governance	24
Risk register	24
Reproducibility controls	25
Governance and responsible use	25
Limitations	25
Recommendations	26
10. Conclusion	26
References	27
Appendix A. Project files	27
Appendix B. Configuration values	28
Appendix C. Traceability identifiers	28
Appendix D. Glossary	28

Executive summary

Every impeller leaving a foundry has to be checked for casting flaws before it is shipped. This project replaces the first pass of that manual check with a convolutional neural network that looks at a top-down photograph of each part and decides whether it is sound (*ok_front*) or flawed (*def_front*). Around the model sits a complete operating framework: quality gates on incoming data, frozen and fingerprinted dataset versions, logged experiments, a model registry with an explicit production pointer, a web service packaged in a container and tested on every commit, and a monitoring loop that can request retraining but cannot release a new model unless it proves itself.

Topic	What the executed notebooks show
Input data	7,348 images checked; none unreadable, all 300 × 300 pixels, no image carries two labels. Content hashing exposed 64 images present in both the original train and test folders; they were removed from training, leaving 7,284 distinct images.
Data version v1	5,583 training, 986 validation and 715 test images; label-stratified, seed 42, with a fingerprint of the full image inventory.
Network	ResNet18 with ImageNet weights; only the new 512 → 2 output layer learns (1,026 of 11,177,538 weights, under 0.01%).
Model selection	Epoch 4 chosen on validation defect F1 = 0.9287 (recall 0.9929, precision 0.8723).
Held-out test	Recall 0.9978 , precision 0.9113, F1 0.9526, ROC-AUC 0.9966, accuracy 0.9371. One defective part missed, 44 sound parts flagged.
Registry	Version 4 of casting_defect_classifier is @production and carries its validation and test scores as tags.
Service and CI	Health and prediction endpoints exercised live, including rejection of bad uploads; 8 automated tests pass; the GitHub Actions pipeline builds the Docker image successfully.
Monitoring	On a deliberately degraded batch, average confidence fell by 0.121, every image feature drifted (PSI > 0.2) and embedding PSI reached 12.43; all three retraining triggers fired.
Release decision	A retrained candidate gained only +0.0094 F1 on degraded images, short of the +0.01 bar, so it was rejected and version 4 kept serving.

Bottom line. Under normal imaging conditions the model comfortably satisfies the business requirement: only one flawed casting in 453 would slip through. When the images are made darker, blurrier and noisier, however, roughly a quarter of the defects are no longer caught, and updating only the final layer does not fix that. The operational safeguards worked: the degradation was detected by three independent signals and a candidate with an unconvincing improvement was kept out of production. The most valuable next investment is to unfreeze the deepest convolutional block and train it on real, labelled images captured under the degraded conditions, and to make defect recall part of the release rule.

1. Introduction

Background

Sand and die casting can leave a range of surface and near-surface flaws: gas porosity (blowholes and pinholes), shrinkage cavities, burrs, flash, cold shuts and cracks. On a pump impeller even a small flaw can unbalance the rotor, erode under flow or start a fatigue crack, so a single escaped part can lead to an in-service failure. Visual inspection by trained staff is still common, but it scales poorly with volume, its quality varies with fatigue and lighting, and it leaves little record of why a part was accepted.

Image classifiers built on convolutional networks have become a practical way to automate this first screening step. Building an accurate classifier is only half the job, though. A factory deployment also has to answer operational questions: which data was the model trained on, can the result be reproduced, which version is live, how will anyone notice if the camera setup changes, and who decides when a replacement model is good enough. Those questions are the subject matter of MLOps, and this capstone answers them alongside the modelling work.

Scope and approach

The project was delivered in four stages, each implemented as reusable Python modules called from a notebook:

Stage	Notebook	Main modules	Outcome
1. Understand and prepare data	Data_Preparation	data_prep.py	Validated, de-duplicated, versioned splits; EDA; transforms
2. Build and track the model	Model_Development_and_Tracking	model.py, train.py	Trained ResNet18 head, MLflow runs and logged model
3. Evaluate and deploy	Model_Development_and_Tracking	evaluate.py, app.py	Test metrics, registry version, API, Docker, CI
4. Operate and govern	Operations_Monitoring_and_Evidence	monitoring.py, retrain.py	Drift signals, triggers, candidate, release decision

All modelling ran on CPU in Google Colab, with project files on Google Drive. The remaining sections follow the order of the assessment stages; each sub-heading carries the corresponding task number.

2. Business and data understanding

1.1.1 Business objective and justification

Goal. Classify every impeller photograph at the end of the casting line as acceptable or defective, so that flawed parts are held back automatically and human inspectors only review the parts the model flags.

The expected benefits are:

- **Fewer escapes:** a consistent detector that does not tire or vary between shifts;
- **Lower inspection effort:** parts judged sound pass without handling, so staff time concentrates on the flagged minority;
- **Traceability:** each decision is logged with a timestamp, probability and model version, giving quality engineers an audit trail;
- **Faster feedback to the foundry:** defect rates can be tracked per batch rather than discovered at the customer.

Why machine learning, and why MLOps. Section 3 shows that simple image statistics (brightness, contrast, edge counts) barely differ between good and bad parts, because flaws cover only a small area of the casting. Hand-written rules on such measurements would not separate the classes, whereas a CNN learns local visual patterns directly. Factory imaging also changes over time: lamps dim, lenses collect dust, fixtures shift. A model that is correct at launch can quietly degrade, so it needs controlled data, reproducible training, live monitoring and a disciplined release process, not just a one-off training run.

1.1.2 Choice of success metric

Defective is treated as the positive class (*def_front*, label 1). The two kinds of mistake have very different consequences:

Mistake	What happened	Consequence on the shop floor	Relative cost
False negative	Flawed part labelled sound	Part is shipped; risk of pump failure, warranty claim, safety incident, lost customer trust	Severe
False positive	Sound part labelled flawed	Part waits for a manual look and is then released	Minor

This asymmetry determines the metric hierarchy:

$$\begin{aligned} \text{Recall}_{\text{defect}} &= TP / (TP + FN) \\ \text{Precision}_{\text{defect}} &= TP / (TP + FP) \\ F1 &= 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \end{aligned}$$

- **Primary metric - recall on def_front.** It answers the question the business cares about most: of all flawed parts, what share did we stop? The target is at least 0.99 on unseen data.
- **Supporting metrics - precision and F1 on def_front.** A detector can reach perfect recall by rejecting everything, so precision is reported to expose the re-inspection burden, and F1 (which balances the two) is used to choose the best checkpoint on validation data.
- **ROC-AUC** summarises ranking quality over every possible threshold and indicates how much room there is to move the 0.5 cut-off.
- **Accuracy is reported but never used for decisions.** With 63.4% of test images defective, a model that always answers 'defective' already scores 63.4%, so accuracy says little about usefulness.

1.1.3 Dataset structure and classes

The images come from the publicly available *casting product image data for quality inspection* collection on Kaggle [1]. They are stored as `casting_data/{train,test}/{ok_front,def_front}/`; every file is a 300×300 JPEG showing a single impeller from above with greyscale content. The label mapping, fixed in `config.py`, is `ok_front` $\rightarrow$ 0 and `def_front` $\rightarrow$ 1.

Folder	ok_front (0)	def_front (1)	Total	Share defective
train	2,875	3,758	6,633	56.7%
test	262	453	715	63.4%
Total	3,137	4,211	7,348	57.3%

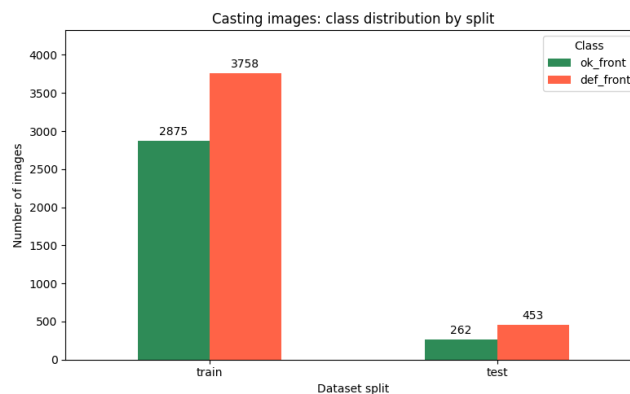


Figure 1. Image counts per class in the original train and test folders.

Unusually for a defect dataset, flawed parts are the larger class. The imbalance is modest, so no over- or under-sampling was applied; instead the loss function is re-weighted (Section 5) so that neither class dominates the gradient.

3. Data quality and dataset versioning

1.2.1 - 1.2.3 Automated quality checks

Before any split is created, `validate_quality()` inspects every file. Integrity is tested twice: PIL's `verify()` detects structural damage, and a full `load()` then forces every pixel to be decoded, which catches truncated files that pass the first test. Each image's size is compared with the expected 300×300 , and a SHA-256 digest of its raw bytes is recorded for duplicate detection.

The checks fall into two groups. **Blocking** checks describe problems that make the data unsafe to use and must all pass. **Review** checks describe issues that are acceptable at this point because the next step corrects them automatically.

Check	Group	Outcome
Images present in all four class folders	blocking	PASS – 7,348 files
Files that are missing or fail to decode	blocking	PASS – none
Images with unexpected dimensions	blocking	PASS – every image is 300×300
Each class readable in each split	blocking	PASS
Same image stored under both labels	blocking	PASS – no conflicts
Byte-identical duplicates	review	64 groups, 64 surplus copies
Duplicate groups that span train and test	review	all 64 $\rightarrow$ removed during splitting

Verdict: every blocking check passed. The class balance per folder is 43.3% / 56.7% (OK / defective) in train and 36.6% / 63.4% in test. The whole report is saved to

artifacts/quality_report.json so that later runs can be compared with it.

Why the duplicates matter. Each of the 64 groups is one photograph saved under the same file name and label in both the train and test folders, for instance `cast_ok_0_1001.jpeg`. Left in place, about one test image in eleven (9.0%) would already have been seen during training. Scores on those images would measure recall of memorised examples rather than the ability to judge new castings, inflating the reported performance. Comparing content hashes, rather than file names, is robust to renaming and never confuses two different photos that happen to share a name. Its blind spot is near-copies: a photo that has been re-compressed, cropped or nudged by a pixel produces a different hash. A perceptual hash would extend the check to those cases.

1.3.1 Reproducible, stratified split

`build_splits(root, 'v1')` turns the validated folders into three disjoint sets in four steps:

- **Collapse duplicates.** For every hash, one copy is kept, and the copy in the test folder is preferred, so the authors' held-out set stays intact and the training folder loses the repeat.
- **Preserve the original test folder** as the final evaluation set. It was assembled separately by the dataset's creators, and keeping it makes results comparable with other work on the same data.
- **Carve a validation set from training:** 15% of the remaining training images, stratified on label, using `random_state = 42`. Records are sorted by path first, so the outcome does not depend on the order in which the operating system lists files.
- **Prove there is no leakage:** the function raises an error if any hash appears in more than one of train, validation and test. The measured overlap is zero.

Split	ok_front	def_front	Total	OK share
train	2,389	3,194	5,583	42.8%
val	422	564	986	42.8%
test	262	453	715	36.6%
All unique images	3,073	4,211	7,284	42.2%

Stratification gives train and validation identical class proportions. The test set keeps its original, more defect-heavy mix. That difference does not affect recall, which is computed within the defective class only, but it does affect precision, which depends on how many sound parts are present. Precision on validation and on test should therefore not be compared one-to-one.

1.3.2 Snapshot and metadata

The split is written to `artifacts/splits/v1/` as five files:

File	Content
train.json, val.json, test.json	One record per image: relative path, numeric label, class name, original folder, SHA-256
excluded_duplicates.json	Each surplus copy that was dropped and the path of the copy that was kept
metadata.json	Version, UTC creation time, dataset fingerprint, seed, validation fraction, class map, positive class, counts before and after de-duplication, split policy, per-split sizes and class counts

```
version: v1          created_utc: 2026-10-07T09:58:27Z      seed: 42
dataset_fingerprint: 907a855bb7e9041cb3cfa856a9d0fbd0750dd562ccca959fb41f44c669496f92
validation_fraction_of_clean_train: 0.15  positive_class: def_front
original_image_count: 7348  unique_image_count: 7284  excluded_duplicate_count: 64
cross_split_hash_overlap: 0
```

The **fingerprint** is a SHA-256 over the sorted list of every image's path, label and hash, so any added, removed, relabelled or edited image changes it. Versions are treated as **read-only**: calling

`build_splits` again on identical data simply returns the stored metadata, while any difference in data or settings raises an error asking for a new version name (for example v2). When the snapshot was reloaded, every split matched its recorded size and class counts, and every listed file existed on disk.

4. Exploratory analysis and workflow design

1.4.1 What the images look like

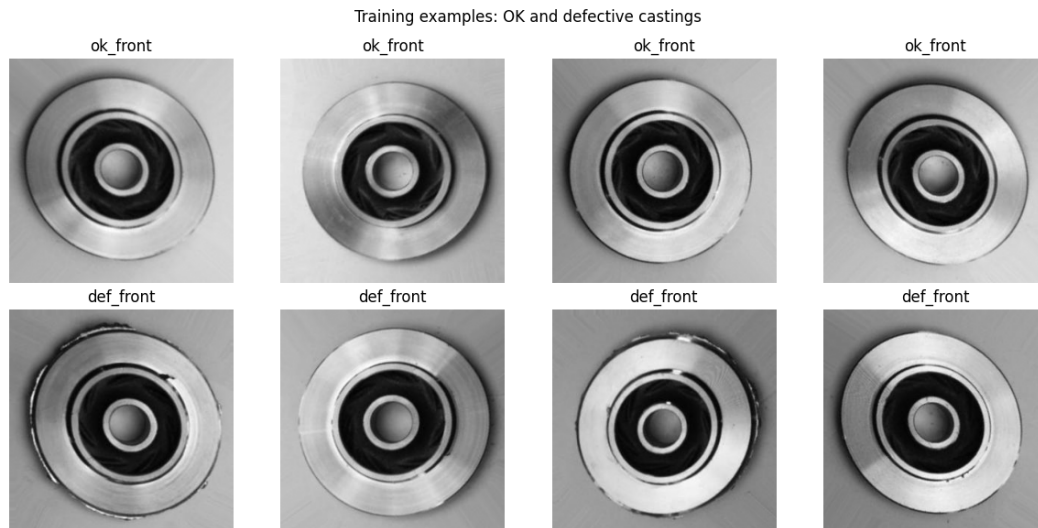


Figure 2. Four randomly drawn training images per class (seed 42). Top: sound; bottom: defective.

Visually the two classes are hard to tell apart at thumbnail size; flaws appear as small dark pits, rough patches or nicks on the rim and blades. To quantify this, `image_features()` measures five whole-image properties on the greyscale pixels of all 5,583 training images. The validation and test sets were deliberately left out of this analysis so that no design choice was influenced by them.

Feature	How it is computed	Sound mean	Defective mean	Separation
brightness	average pixel value (0-255)	149.9	139.3	Good – interquartile ranges hardly meet
contrast	standard deviation of pixels	62.3	58.6	Partial – sound parts slightly higher
edge_density	fraction of pixels with gradient magnitude > 20	0.072	0.079	Weak – defective spread is wider
sharpness	variance of a 4-neighbour Laplacian	170.5	192.3	Weak – large overlap
dark_ratio	fraction of pixels darker than 50	0.134	0.139	Weak – large overlap

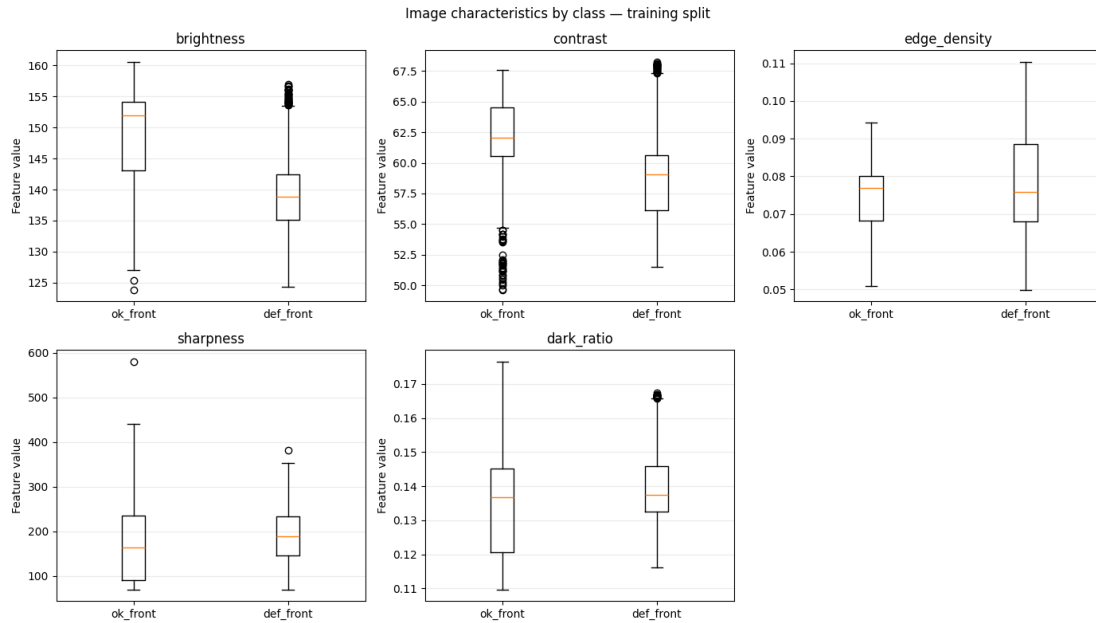


Figure 3. Box plots of the five measurements for each class (training split).

Findings and the decisions they led to

- **Global statistics cannot do the job.** Four of the five measures overlap almost entirely between classes. This is expected: a blowhole changes a few hundred pixels out of 90,000. A model that looks at local spatial structure, such as a CNN, is required.
- **Brightness is suspicious.** Flawed parts are on average about 10 grey levels darker, and this is the one feature that separates the classes reasonably well. A local flaw cannot darken a whole image by that much, so the gap more likely comes from how batches were photographed. A network could exploit it as a shortcut and then fail when the lighting changes. Two counter-measures follow: random brightness and contrast changes during training, and brightness and contrast as monitored quantities in production.
- **Inputs are uniform.** One size, one orientation, greyscale content, part centred. Resizing to 224 pixels and copying the grey channel into three channels is all that is needed to match the pretrained network, and mirror images, small rotations and small shifts are realistic variations that do not change the label.
- **Reusable for monitoring.** The same function is later run on incoming images to compare their distributions with a clean reference.

1.4.2 End-to-end MLOps workflow

The workflow was designed as a closed loop rather than a straight line, so that evidence from production can flow back into training under controlled conditions.

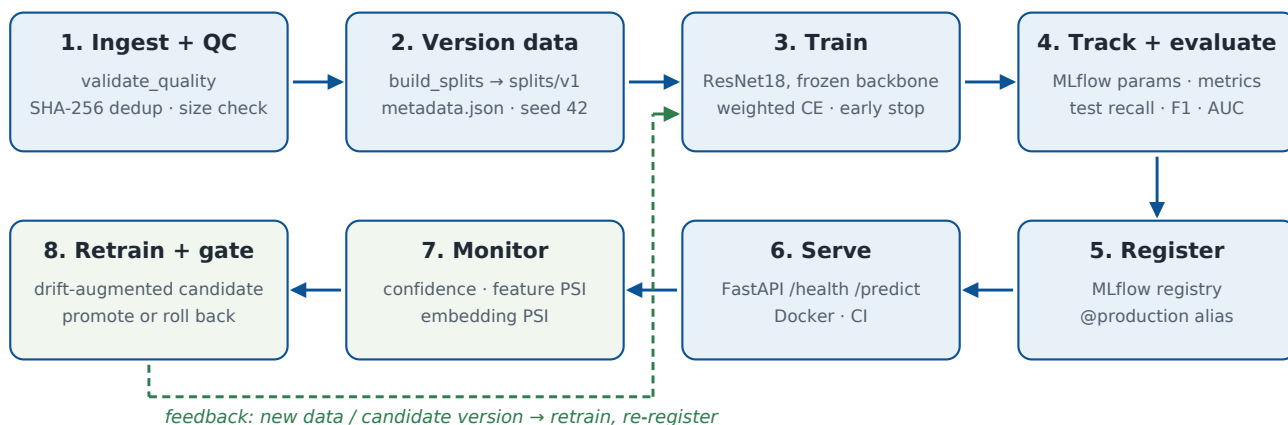


Figure 4. Lifecycle of the solution. Solid arrows: build-and-release path. Dashed arrow: monitored feedback that can lead to retraining.

Step	Implementation and artefacts	Control it provides
Ingest + QC	validate_quality → quality_report.json	Unsafe data is stopped before it reaches a split
Version	build_splits → splits/v1/*.json + metadata.json	Fixed, fingerprinted, leakage-checked datasets
Train	python -m src.train	Seeded run; best epoch chosen on validation only
Track + evaluate	MLflow experiment casting_defect_detection; src/evaluate.py	Every parameter, curve, model and test result is recorded
Register	MLflow model casting_defect_classifier, alias @production	Single pointer to the live model; instant rollback
Serve	app.py, Dockerfile, .github/workflows/ci.yml	Validated requests, prediction log, tested and buildable on every push
Monitor	src/monitoring.py → drift_summary.json, drift_report.html	Three label-free health signals
Retrain + approve	src/retrain.py → retraining_decision.json	Explicit triggers and a promotion rule with a safety margin

Design decisions and alternatives considered.

Decision	Chosen option	Alternative	Reason for the choice
Backbone	ResNet18, ImageNet weights [2], [3]	Larger nets (ResNet50, EfficientNet)	Small, fast on CPU, well understood; adequate for 224 px greyscale parts
Training scope	Frozen backbone, new head only	Full fine-tuning	~5.6k images: low overfitting risk, minutes per run; deeper tuning kept as next step
Split	Keep original test folder	Pool all images and re-split	Comparability with other work; avoids reshuffling the authors' holdout
Imbalance	Inverse-frequency class weights	Over/under-sampling	Mild imbalance; weighting keeps every image and needs no resampling logic
Tracking + registry	MLflow with aliases [4]	Spreadsheet / folder naming	Searchable runs, lineage from run to served model, one-step rollback
Serving	FastAPI in a CPU Docker image [5]	Batch scoring script	Matches an inline station that scores one part at a time

Decision	Chosen option	Alternative	Reason for the choice
Drift signals	PSI on features and embeddings + confidence [6], [7]	Labelled accuracy only	Labels arrive late on the line; these signals need none

5. Preprocessing, model, training and tracking

2.1.1 Preprocessing for the pretrained network

`get_transforms(train=False)` defines the fixed pipeline used for validation, test and the live API. `get_transforms(train=True)` inserts random augmentation in the middle of the same pipeline:

Evaluation / serving (no randomness) Grayscale(num_output_channels=3) Resize((224, 224)) ToTensor() # 0–255 → 0–1 Normalize(ImageNet mean, std)	Training (random, label-preserving) Grayscale(3) → Resize(224, 224) RandomHorizontalFlip(p=0.5) RandomAffine(±15°, shift ±5%, fill=128) ColorJitter(brightness ±20%, contrast ±20%) ToTensor() → Normalize(ImageNet)
--	--

- **Three channels:** ResNet18 expects RGB input, so the single grey channel is replicated three times rather than retraining the first convolution.
- **224 × 224:** the input size the ImageNet weights were trained at.
- **ImageNet normalisation** (mean 0.485/0.456/0.406, std 0.229/0.224/0.225): shifts pixel values into the range the pretrained filters expect, which matters for transfer learning to work well.
- **Automated checks** confirmed that the output is a finite float32 tensor of shape (3, 224, 224), that two passes of the evaluation pipeline give bit-identical output, and that the normalised values equal $(x - \text{mean}) / \text{std}$ exactly.

2.1.2 Augmentation for training only

- **Horizontal flip (50%)** and **rotation up to ±15° with up to 5% shift** imitate small differences in how a part is placed under the camera. Corners exposed by the rotation are filled with mid-grey (128).
- **Brightness and contrast jitter of ±20%** directly targets the lighting shortcut found in the EDA, forcing the network to rely on the flaw rather than the exposure.
- None of these operations can turn a sound part into a flawed one or the reverse, so labels stay valid.
- Validation, test and serving receive **no** augmentation, so every reported metric is repeatable and reflects images exactly as the camera produces them.
- *Accepted cost:* shrinking 300 px to 224 px reduces very small flaws by about a quarter in each direction.



Figure 5. Left to right: source image, evaluation pipeline output, three random training variants.

2.2.1 - 2.2.2 Transfer-learning architecture

ResNet18 [2] is an 18-layer residual network whose skip connections make deep CNNs easy to optimise. Its ImageNet-trained convolutional layers [3] already respond to edges, textures and blobs, which are exactly the cues that describe casting flaws. The network is reused as follows:

Component	Treatment	Parameters
Convolutional backbone (conv1 ... layer4, BatchNorm)	Loaded with IMAGENET1K_V1; requires_grad = False; kept in eval() mode	11,176,512 (frozen)
Global average pooling	Unchanged; produces a 512-value descriptor per image	0
Output layer fc	Replaced: Linear(512 → 2), randomly initialised and trained	1,026 (trainable)
Whole model		11,177,538

- Keeping BatchNorm layers in evaluation mode during training stops their running averages from being re-estimated on the small casting set, which would otherwise subtly change the frozen features.
- The notebook asserts that only `fc.weight` and `fc.bias` require gradients and that exactly these two tensors reach the optimiser.
- `EmbeddingExtractor` exposes the 512-value descriptor before the output layer; Stage 4.3 uses it to watch for changes in what the network 'sees'.

2.3.1 Loss, optimiser and seeding

Class weights follow the inverse-frequency rule, where N is the number of training images, K the number of classes and n_c the count of class c :

$$w_c = N / (K \cdot n_c)$$
$$w_{OK} = 5,583 / (2 \cdot 2,389) = 1.1685; w_{defect} = 5,583 / (2 \cdot 3,194) = 0.8740$$

- These weights make the two classes contribute equally to the cross-entropy loss. Because defective parts are the majority here, the weighting slightly favours the OK class; recall on defects is protected instead by checkpoint selection and, if needed later, by the decision threshold.
- **Optimiser:** Adam [8] over the two head tensors only; learning rate 1×10^{-3} , weight decay 1×10^{-4} , mini-batches of 32.
- **Determinism:** seed 42 is applied to Python, NumPy and PyTorch; the shuffling `DataLoader` receives its own seeded generator; cuDNN runs in deterministic mode with benchmarking disabled.

2.3.2 Epoch loop, validation and early stopping

Each epoch trains on augmented images and then scores the untouched validation set, recording loss, F1, precision and recall for the defect class. The weights with the highest validation F1 are copied aside; if F1 fails to improve for three consecutive epochs the loop ends. At the end the best weights are restored and written to `artifacts/model.pt`, together with `model_meta.json` (history, class weights, best epoch, MLflow run id).

Epoch	Train loss	Train F1	Val loss	Val F1	Val precision	Val recall
1	0.4194	0.8555	0.2983	0.9241	0.8642	0.9929
2	0.2586	0.9325	0.2928	0.8857	0.7972	0.9965
3	0.1965	0.9534	0.2322	0.9218	0.8602	0.9929

Epoch	Train loss	Train F1	Val loss	Val F1	Val precision	Val recall
4 (selected)	0.1620	0.9630	0.2133	0.9287	0.8723	0.9929

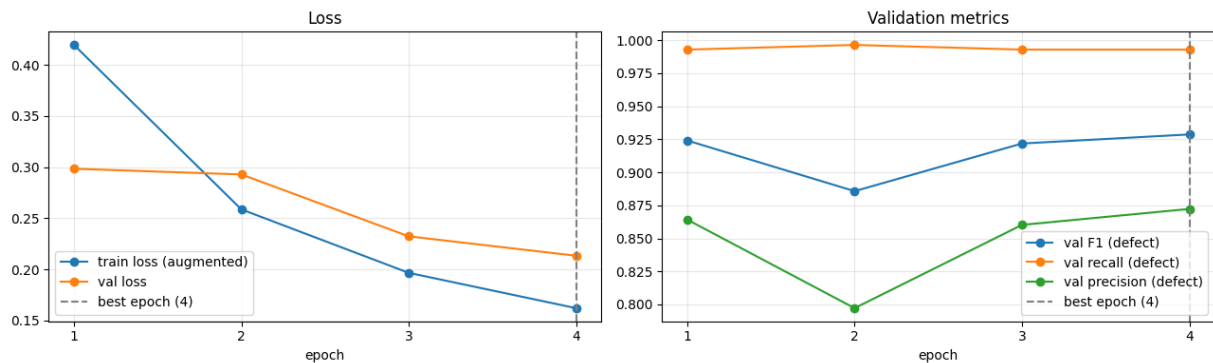


Figure 6. Loss and validation metrics per epoch; the dashed line marks the restored epoch.

Validation recall stays at 0.993–0.997 throughout; what improves is precision, i.e. the head learns to stop flagging sound parts. The dip at epoch 2 is a precision drop (0.797) with recall at its highest, which is typical of a linear head whose boundary is still moving. The configured budget was four epochs and F1 was still rising at the end, so early stopping never fired; a longer run is a cheap experiment. Training loss sits above validation loss early on because it is measured on augmented, harder images.

2.4.1 - 2.4.2 Experiment tracking with MLflow

Training logs to the MLflow [4] experiment casting_defect_detection, stored on Google Drive. The production run has id ccb03cd91af0447aaf0a10cdbacd714e and the name resnet18_v1. The notebook reads every item back from MLflow to prove that it was stored.

Category	Recorded content
Parameters (20)	data version, seed, image size, train and validation counts, train class counts; backbone, pretrained weights, frozen flag, total and trainable parameters; optimiser, learning rate, weight decay, batch size, epoch budget, stopping metric and patience, loss type, class weights
Curves (step = epoch)	train_loss, train_f1, val_loss, val_f1, val_precision_defect, val_recall_defect
Summary values	best_epoch 4, best_val_f1 0.9287, best_val_precision_defect, best_val_recall_defect 0.9929, epochs_completed; later test_recall_defect, test_precision_defect, test_f1_defect, test_roc_auc, test_accuracy
Model	mlflow.pytorch.log_model → LoggedModel m-58cde3d446ee41199f9a3919283d2b37 (status READY); signature: float32 tensor (−1, 3, 224, 224) in, float32 (−1, 2) out; input example attached
Files	checkpoint/model.pt, history.json, metadata/model_meta.json, evaluation/ (metrics.json, model_eval.png, failure_cases.json/png, training_curves.png)

Evidence of reproducibility. The experiment holds four completed runs of the same configuration, started between 7 October 19:30 and 8 October 14:20 UTC. Each reports exactly the same best validation F1 (0.92869) and recall (0.992908) at epoch 4, which confirms that the seeding makes training repeatable bit-for-bit on the same platform. Three further runs were interrupted during development (two FAILED, one left RUNNING); none of them is linked to a registered version.

6. Evaluation, registry and deployment

3.1.1 Test-set metrics and model selection

Model selection used validation data only. The 715 test images were scored once, after selection, so the figures below are an unbiased estimate of performance on new parts. Because duplicates were removed, none of these images was available during training.

Measure (positive = def_front)	Value	Meaning
Recall	0.9978	452 of 453 defective parts stopped
Precision	0.9113	452 of 496 flagged parts were truly defective
F1	0.9526	Balance of the two above
ROC-AUC	0.9966	Ranking quality across all thresholds
Specificity	0.8321	218 of 262 sound parts passed without review
Negative predictive value	0.9954	Of 219 parts passed, 218 were truly sound
Balanced accuracy	0.9149	Mean of recall and specificity
Matthews correlation	0.867	Single summary robust to class imbalance
Accuracy	0.9371	Compare: 0.6336 by always answering 'defective'

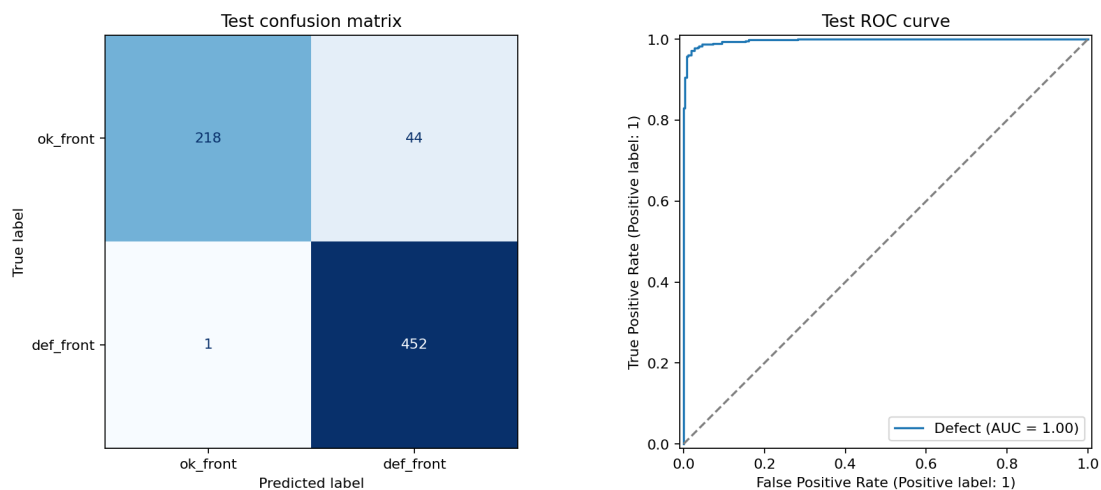


Figure 7. Confusion matrix and ROC curve on the 715 held-out test images.

What this means operationally. Applying the test rates to 1,000 castings with the test set's defect mix (about 634 defective, 366 sound):

Outcome per 1,000 castings	Expected count
Defective parts stopped	≈ 632
Defective parts shipped (missed)	≈ 1.4
Sound parts sent for manual review	≈ 62
Total parts routed to an inspector	≈ 694

The recall target of 0.99 is met with a wide margin. The price is the false-alarm rate: 16.8% of sound parts go to manual review. With a ROC-AUC of 0.997 the ranking is close to perfect, so moving the 0.5 cut-off (chosen on validation data, never on test) is a realistic lever for reducing that load while keeping recall above 0.99.

3.1.2 Failure analysis

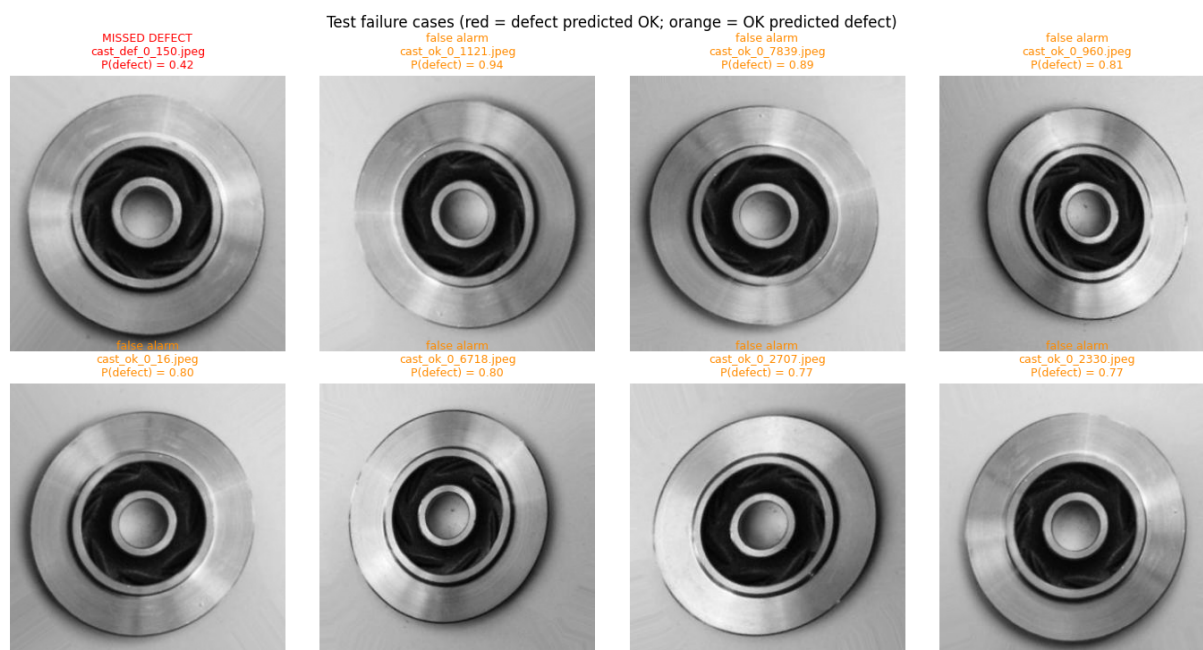


Figure 8. The model's most confident mistakes. Red title: missed defect; orange title: false alarm.

All 45 errors were exported to `failure_cases.json`, ordered by how far their probability was from 0.5. Reviewing them shows:

- **The single miss**, `cast_def_0_150.jpeg`, received $P(\text{defect}) = 0.42$. It sits just on the wrong side of the boundary rather than being confidently wrong, so a modestly lower threshold would capture it.
- **False alarms are confident** ($P(\text{defect})$ between 0.68 and 0.94 for the 15 strongest). At this resolution most look clean to the eye; several have a darker rim or uneven illumination, consistent with the lighting shortcut predicted in the EDA.
- **No obvious labelling errors** were spotted among the top mistakes. Having a quality engineer review the 44 false alarms is recommended before any threshold change, in case some are genuine but subtle flaws.

3.2.1 - 3.2.2 Registration, promotion and version history

`register_and_promote()` reads the run id from `model_meta.json`, which guarantees that the registered model is exactly the checkpoint the API serves. It finds the `LoggedModel` produced by that run, registers it under **casting_defect_classifier**, and points the **@production** alias at the new version. Aliases are MLflow's current replacement for the older 'Staging/Production' stages. Calling the function again for the same run re-uses the existing version instead of creating a duplicate.

Version	Registered (UTC)	Source run	Alias	Val F1	Test recall	Status
1	2026-10-07 20:45	64349466...		0.9287	—	historical
2	2026-10-08 05:34	89164524...		0.9287	0.9978	historical
3	2026-10-08 12:11	e6b64617...		0.9287	—	historical
4	2026-10-08 15:19	ccb03cd9...	@production	0.9287	0.9978	live
5, 6	Stage 4.4	retraining runs		—	—	rejected_rollback

Version 4 is annotated with the reasons for its promotion: data version v1, the selection rule (highest validation defect F1 with early stopping), best epoch 4, validation F1 0.9287 and recall 0.9929, and the test results (recall 0.9978, precision 0.9113, F1 0.9526, ROC-AUC 0.9966). Because earlier versions are never deleted, reverting is a single call, `set_registered_model_alias(name, 'production', <version>)`, with no retraining. The full history is also exported to `model_registry_history.json`.

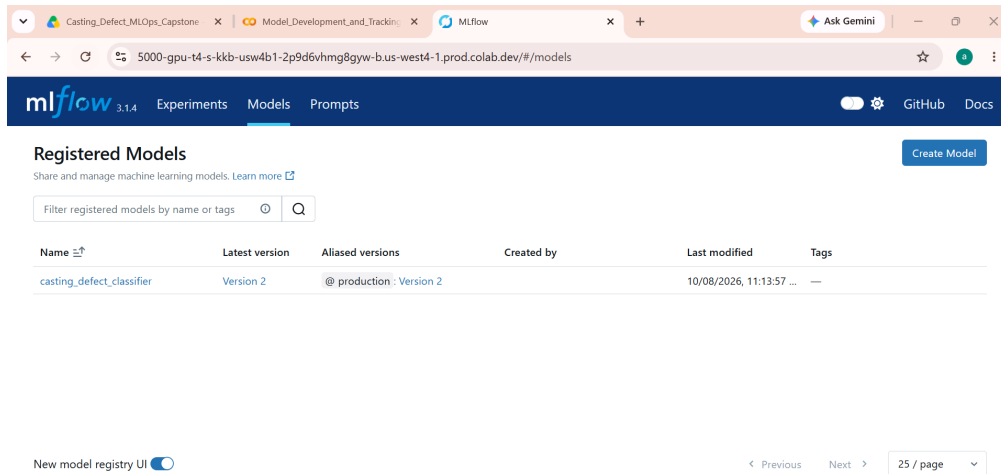


Figure 9. MLflow Models page with `casting_defect_classifier` and its `@production` alias. The screenshot dates from earlier in development, when version 2 held the alias; the table above shows the final state, with version 4 in production.

Workaround applied. With MLflow 3.1.4's file-based store, saving a description on a version that has metrics attached failed with a `RepresenterError`. The code catches the error and saves the same text as a `description` tag, so no information is lost.

3.3.1 Inference service

`app.py` is a FastAPI [5] application. At start-up a lifespan hook loads the checkpoint, the evaluation transform and the metadata once, so individual requests do no disk reads of the model. Its behaviour:

Route / case	Behaviour
GET /health	Liveness plus model facts: <code>model_loaded</code> , class map, positive class, registry version, run id, test metrics
POST /predict	Accepts one image as multipart UploadFile; returns label, <code>is_defective</code> , <code>prob_defect</code> and confidence
Bad input → 400	Empty upload, file over 10 MB, or anything that cannot be fully decoded as an image (including fake or truncated JPEGs)
No model → 503	Service still starts (useful in CI); <code>/health</code> reports <code>model_loaded = false</code>
Logging	Every prediction becomes one JSON line in <code>predictions.log</code> and a <code>casting_api</code> log record with timestamp, file name, model version and latency

Requests go through exactly the same preprocessing as validation and test, which removes the classic training-serving skew. The full image is decoded before inference, so a corrupt file is rejected with a clear 400 instead of crashing the worker with a 500.

3.3.2 Live demonstration

FastAPI's `TestClient` runs the real application in-process, including the start-up hook:

Request	HTTP	Response body (abridged)
GET /health	200	model_loaded true, model_version 4, run ccb03cd9..., test recall 0.9978
POST /predict - cast_ok_0_10.jpeg (sound)	200	label ok_front, prob_defect 0.4288, confidence 0.5712
POST /predict - cast_def_0_1059.jpeg (defective)	200	label def_front, prob_defect 0.8501, confidence 0.8501
POST /predict - plain-text file	400	Upload must be a valid image file.
POST /predict - fake JPEG bytes	400	Upload must be a valid image file.

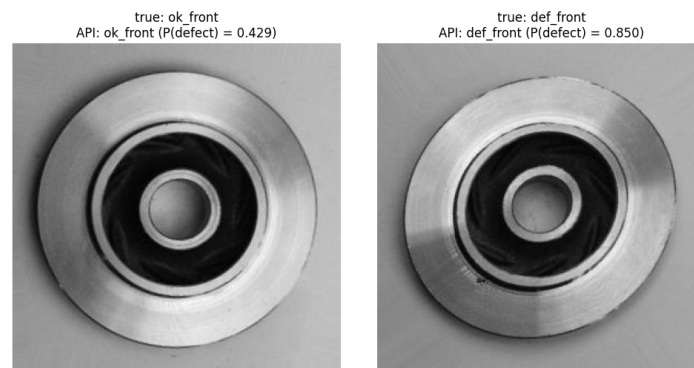


Figure 10. The two test photographs submitted to the running API, with its answers.

Both answers are right, and both rejections behave as intended. The sound part was classified with only 57% confidence, which is in line with the false-alarm tendency on sound parts. Each request took around 110 ms on CPU, comfortably within the cycle time of a typical inspection station. The captured log records show one start-up message, two prediction records and two rejection records.

3.4.1 Container image

```
FROM python:3.11-slim
RUN apt-get install libgl1 libglv2.0-0 # runtime libraries for image I/O
RUN pip install --index-url ../whl/cpu torch==2.8.0 torchvision==0.23.0 \
    && pip install mlflow==3.1.4 fastapi==0.128.8 uvicorn==0.39.0 python-multipart==0.0.20 \
    evidently==0.7.20 scikit-learn==1.6.1 pillow==11.3.0 numpy pandas matplotlib
COPY config.py app.py ./ ; COPY src ./src
COPY artifacts/model.pt artifacts/model_meta.json ./artifacts/
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

- A slim Python base and CPU-only PyTorch wheels keep the image far smaller than a CUDA build; inference on one image per request does not need a GPU.
- Every library is pinned to the version used in development, so the container behaves like the notebook.
- Only serving files are copied: configuration, API, source package, weights and metadata.

3.4.2 Continuous integration

.github/workflows/ci.yml runs on each push to *main* and on every pull request. It installs Python 3.11 and the CPU dependencies, runs `pytest -q`, and finally builds the Docker image. A failing test or a broken image therefore blocks the change.

- **Local test run:** 8 tests passed in 18.14 s.
- **Pipeline run:** GitHub Actions workflow *ci*, run #1 on commit `cebcf59`, finished green in about 3 minutes; the 'Validate Docker build' step completed all eight layers and tagged `casting-defect-api:ci` (1 min 39 s).
- Colab provides no Docker daemon, so the image build is evidenced by the CI run rather than the notebook.

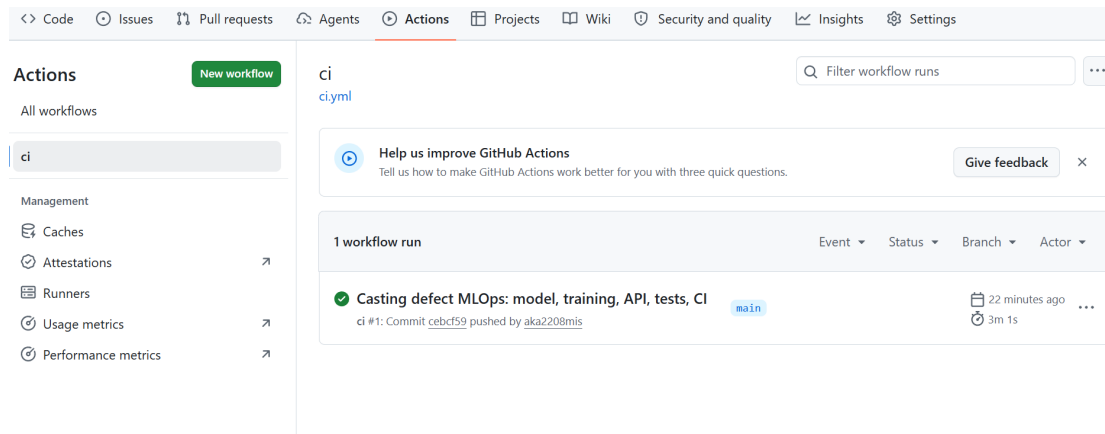


Figure 11. Successful GitHub Actions run of the ci workflow.

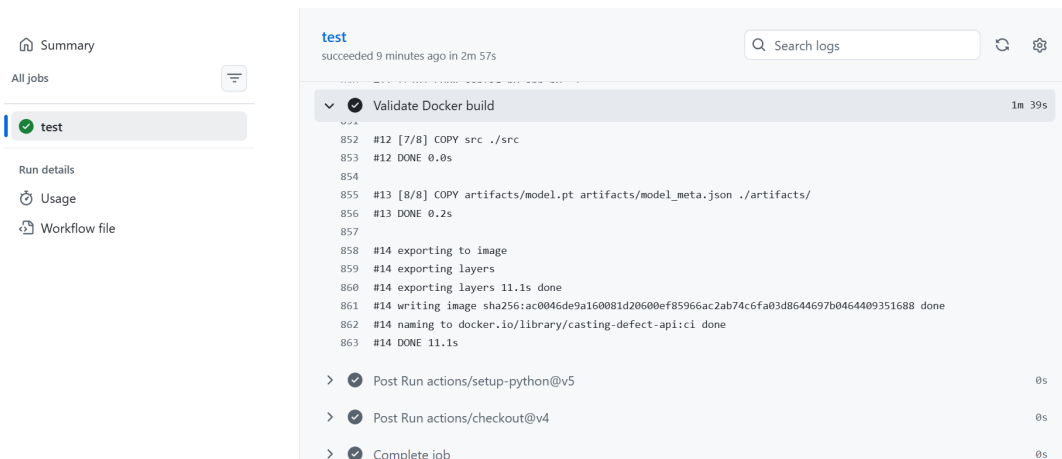


Figure 12. Log of the Docker build step inside the CI job.

Improvements identified during review

- Add artifacts/metrics.json to the image so that /health inside the container also reports test metrics.
- Install from requirements.txt in both the Dockerfile and the workflow; versions are currently listed in three places and could drift apart.
- Run as a non-root user and declare a HEALTHCHECK that calls /health.
- The build relies on the 45 MB checkpoint being committed to Git. For larger models, download the @production version from the registry at build or start time instead.

7. Monitoring and drift detection

In production, ground-truth labels for each casting arrive late or not at all, so the model's health has to be inferred from its inputs and outputs. Three complementary signals are used, all computed by `src/monitoring.py` and stored in `drift_summary.json`.

Test scenario. A reference batch of 300 clean images from the v1 test split is compared with a 'current' batch made from the same 300 images after a seeded degradation (`config.DRIFT_SIM`) that imitates common faults at an inspection station:

Degradation	Setting	Real-world cause it represents
Darkening	brightness × 0.6	Ageing or failing lamp
Gaussian blur	radius 1.5	Dirty or out-of-focus lens
Additive noise	$\sigma = 12$ grey levels	Noisy sensor or higher camera gain
Rotation	fixed 8° turn	Part seated off-angle in the fixture

4.1.1 Prediction logging

Every call to `/predict` appends a JSON line to `artifacts/predictions.log`. All entries carry a timestamp, and the log contains predictions from both version 2 and version 4, which shows that the record identifies the exact model behind each decision:

```
{ "timestamp": "2026-10-08T15:28:51+0000", "filename": "cast_def_0_1059.jpeg",  
  "model_version": "4", "latency_ms": 110.68, "label": "def_front",  
  "is_defective": true, "prob_defect": 0.850093, "confidence": 0.850093 }
```

4.1.2 Confidence monitoring. Confidence is the probability the model assigns to the class it predicts. When inputs move away from what the model was trained on, it typically becomes less certain, so a fall in average confidence is an early, label-free warning.

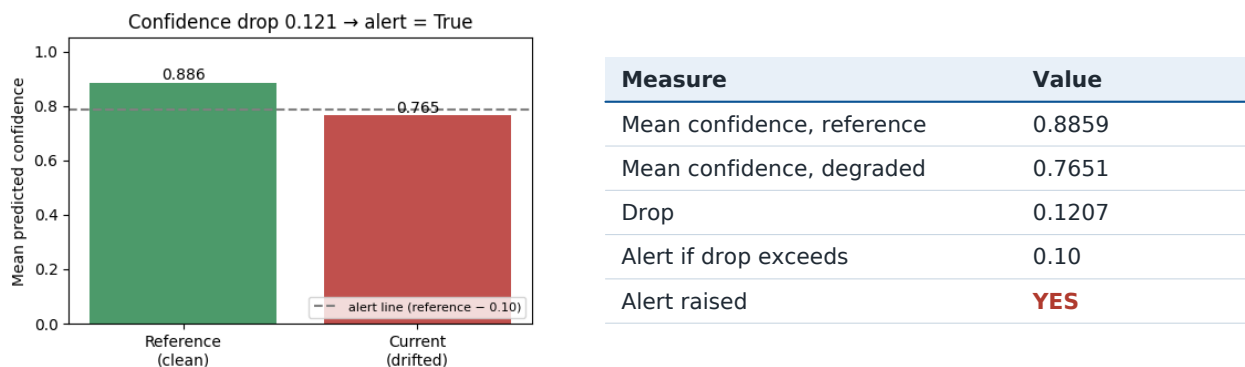


Figure 13. Average confidence before and after degradation, with the alert level.

The limitation of this signal is that a model can remain sure of itself while being wrong, as the confident false alarms in Figure 8 illustrate. It is therefore never used on its own.

4.2.1 - 4.2.2 Feature drift measured with PSI

The five monitored measurements (`config.DRIFT_FEATURES`: brightness, contrast, edge_density, sharpness, mean_intensity) are computed for each image in both batches and compared with the Population Stability Index [6]. The reference values are cut into 10 bins at their deciles, and the share of each batch in each bin is compared:

$$PSI = \sum_{bins} (q_i - p_i) \cdot \ln(q_i / p_i)$$

p = reference share, q = current share, empty bins floored at 10^{-6}

Common practice reads PSI below 0.10 as stable, 0.10–0.20 as a moderate shift and above 0.20 as significant; the project uses 0.20 as the per-feature drift threshold.

Feature	Ref. mean	Degraded mean	Change	PSI	Explanation
brightness	142.36	83.45	−41.4%	12.43*	Images darker, as the lamp fault intends
contrast	59.46	35.73	−39.9%	12.43*	Darkening and blur compress the tonal range
edge_density	0.0756	0.0920	+21.7%	7.51	Noise creates many small false edges
sharpness	233.2	2,793.7	+1,098%	12.43*	Rises although the image is blurred: noise dominates the Laplacian
mean_intensity	142.36	83.45	−41.4%	12.43*	Identical quantity to brightness

* With 10 bins and $\epsilon = 10^{-6}$ the largest attainable PSI is 12.43, reached when none of the degraded values falls inside any reference bin they used to share.

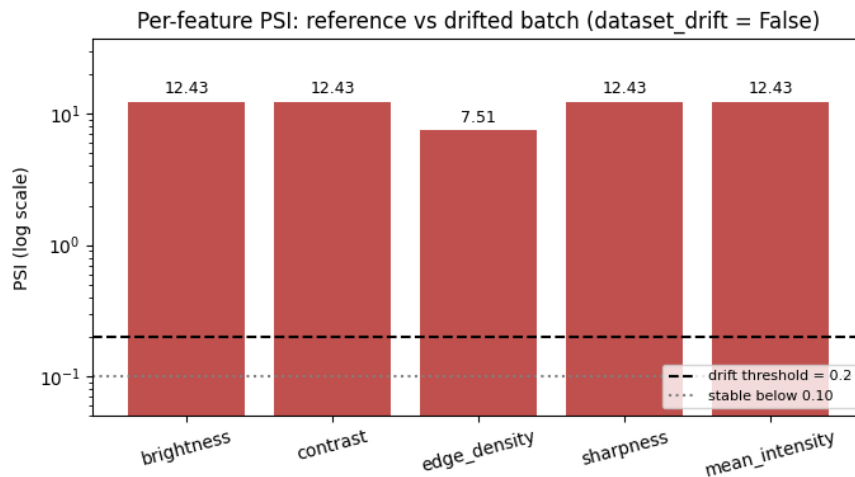


Figure 14. PSI of each feature on a logarithmic axis, with the 0.2 drift line.

Result: all five features exceed the threshold, a drift share of 100% against a trigger level of 40%.

- Four features hit the PSI ceiling, so their exact values carry no ranking information; they simply say the two distributions no longer overlap. Real drift usually builds up gradually and would produce readable values in the 0.1–2 range, which is where PSI is most useful as an early warning.
- The sharpness result is a reminder that a feature can move *against* intuition: blur should lower it, yet added noise raises it tenfold. Thresholds alone are not enough; each alarm needs a physical explanation.
- Because brightness and mean_intensity are the same number, one physical cause is counted twice. Swapping mean_intensity for dark_ratio would make the drift share a fairer summary.

About Evidently. The brief also asks for Evidently's DataDriftPreset [7]. In the final execution of the Operations notebook the monitoring script printed *'Evidently: failed - ModuleNotFoundError: No module named evidently'*: the Colab runtime had been restarted without reinstalling requirements.txt. The script fell back to its PSI test, which produced the 100% drift share and the retraining decision, and the Evidently dataset-drift flag was not computed. Reinstalling `evidently==0.7.20` and re-running `python -m src.monitoring` would add the Evidently report. Given that the distributions do not overlap at all, its per-column tests would be expected to flag every feature too.

4.3.1 Embedding extraction

Pixel statistics describe the image; they do not show whether the *model's* view of the image has changed. To capture that, both batches are passed through `EmbeddingExtractor`, the trained network with its output layer removed, giving one 512-value vector per image. The two matrices have shape (300, 512) each, and the reference embeddings and their mean (the centroid) are stored in `reference_embeddings.npz`.

4.3.2 Drift in feature space

Comparing 512-dimensional clouds directly is awkward, so each vector is reduced to one number: its Euclidean distance from the reference centroid. Images like the training data sit close to the centre; unusual images sit further out. PSI is then computed between the two distance distributions, with an alert above 0.10.

Measure	Clean reference	Degraded batch
Average distance to reference centroid	7.39	12.64 (1.71 × further)
Embedding PSI (alert above 0.10)		12.43 - drift ; identical to <code>drift_summary.json</code>

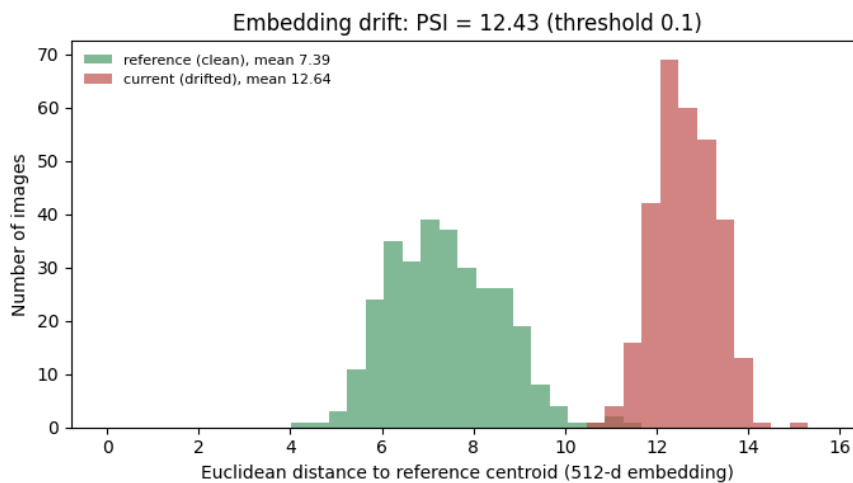


Figure 15. Distance of each embedding from the reference centroid, clean versus degraded.

How this differs from feature drift. Feature PSI reports that the photographs changed; embedding PSI reports that the network's internal description of them changed, which is what actually drives its decisions. The two can disagree in useful ways. A new defect type or a redesigned part might leave brightness and contrast untouched but move the embeddings, while a harmless change in exposure might shift pixel statistics without disturbing the embeddings. In this scenario both signals agree, and the comparison in Section 8 confirms that the change in representation did hurt accuracy.

8. Retraining and release governance

4.4.1 Measurable triggers

`retrain.evaluate_triggers()` turns the monitoring output into three yes/no tests. A retraining job starts if **at least one** is true:

Trigger	Rule	Observed	Limit	Fired
Feature drift share	fraction of features whose PSI exceeds 0.2	1.00	≥ 0.40	Yes
Embedding drift	PSI of distance-to-centroid	12.43	> 0.10	Yes
Confidence drop	reference minus current mean confidence	0.121	> 0.10	Yes

Each signal covers a blind spot of the others: confidence can stay high while wrong; feature PSI can react to changes the network ignores; embedding PSI is tied to the network but is harder to explain to a process engineer. Using an 'any of three' rule favours early detection, which suits a setting where missed defects are costly.

4.4.2 Drift-aware candidate model

When triggered, `python -m src.retrain` builds a challenger:

Setting	Value
Starting point	Production weights (warm start); backbone still frozen, head retrained
Training images	1,500 drawn from v1 train with class proportions preserved (seeded)
Degraded share	40% of training and of validation images pass through the same degradation as monitoring
Schedule	3 epochs, class-weighted loss, Adam, seed 42; best epoch kept by mixed-validation F1
Validation F1 (40% degraded)	0.8891 → 0.9094 (epoch 2, kept) → 0.9006
Tracking	MLflow run <code>retrain_drift_augmented</code> : parameters, per-epoch metrics, scores for both models, checkpoint, model with signature
Registry	Registered as casting_defect_classifier version 6 , tagged with parent version 4, fired triggers and scores

The retraining job ran twice in the course of the project. The first pass created version 5, which was also rejected; the pass shown in the notebook output created **version 6**, the candidate analysed below. Both remain in the registry with the tag `rejected_rollback`. One provided file, `config.py`, was modified to set `MLFLOW_ALLOW_FILE_STORE=true` so that MLflow would accept the local file-based store.

4.5.1 Champion versus challenger

Both models were scored on the complete 715-image test split, once as captured and once with every image degraded. The degradation was seeded, so both models saw identical pixels.

Test condition	v4 F1	v6 F1	Δ F1	v4 recall	v6 recall
Degraded (all images)	0.8110	0.8204	+0.0094	0.7483	0.7461
Clean	0.9526	0.9718	+0.0192	0.9978	0.9890

4.5.2 Promotion rule and decision

A challenger replaces the champion only when it clears **both** conditions, with margin $\epsilon = \text{PROMOTE_EPSILON} = 0.01$:

Condition	Calculation	Result
Clearly better on degraded images	$0.8204 \geq 0.8110 + 0.01 = 0.8210$	Not met
No meaningful loss on clean images	$0.9718 \geq 0.9526 - 0.01 = 0.9426$	Met
Decision	Both needed	Rollback - v4 remains @production

retraining_decision.json records the trigger values, both score sets, the candidate's training summary, the evaluation set, the rule, the reason and the registry history. Had the challenger passed, the script would have archived the current weights, copied the candidate into model.pt, moved the alias and tagged version 4 as *superseded*. Final registry state: versions 1–3 historical, **version 4 live**, versions 5 and 6 rejected.

Discussion

- **The margin did its job.** A gain of under one F1 point on a single synthetic batch is within the range that random variation could produce. Requiring a clear improvement prevents churn between models that are effectively equal.
- **Recall is the real issue.** On degraded images version 4 catches only 0.748 of defects – about 114 of 453 would ship. Version 6 is no better (0.746) and is slightly worse on clean images (0.989 versus 0.998). Because the rule looks only at F1, a candidate that traded recall for precision could in principle pass; adding a recall condition would close that gap.
- **Root cause.** Retraining touches only 1,026 head weights. The frozen convolutional layers were never exposed to dark, blurred, noisy photos and cannot adapt their filters; a new linear boundary over unchanged features has little room to recover.

9. Risk, reproducibility and governance

Risk register

Risk	Likelihood	Impact	Mitigation in place	Further action
Defect escapes under changed imaging	Medium	High	Three drift triggers, confidence alert	Fine-tune layer4 on real degraded images; recall in release rule
Model relies on lighting shortcut	Medium	Medium	Brightness/contrast jitter, brightness monitored	Standardise lighting; inspect saliency maps
High re-inspection workload	High	Low-Med	Precision reported and tracked	Threshold tuning on validation with recall floor
Leakage inflates reported results	Low	High	Hash de-duplication, overlap assertion	Perceptual hashing for near-copies
Wrong model served	Low	High	Alias + version in /health and every log line	Pull @production from registry at deploy time
Unreviewed model reaches production	Low	High	Promotion rule with margin; decision record	Human sign-off step for promotions
Environment differences	Low	Medium	Pinned versions, Docker, CI	Single dependency file for all three

Reproducibility controls

Area	Control
Data	Read-only version v1, inventory fingerprint 907a855b..., per-image SHA-256, refusal to overwrite changed data
Randomness	Seed 42 for splitting, Python, NumPy, PyTorch, the DataLoader, the augmentation demo, drift simulation and retraining; deterministic cuDNN
Code	Notebook cells write the src/ modules; each write is syntax-checked and the previous file backed up to artifacts/backups/src/
Environment	torch 2.8.0, torchvision 0.23.0, MLflow 3.1.4, FastAPI 0.128.8, scikit-learn 1.6.1, Pillow 11.3.0 pinned in requirements, Dockerfile and CI
Lineage	metrics.json and model_meta.json carry the MLflow run id; the API and every logged prediction carry the registry version
Proof	Four separate training runs gave identical validation F1 (0.92869); 8/8 tests pass; CI green

Governance and responsible use

- **Human in the loop.** The model routes parts; it does not scrap them. Every flagged part is seen by an inspector, and periodic audits of passed parts are advised to estimate the true escape rate.
- **Accountability.** Each version records who/what produced it (run id), on which data (version and fingerprint) and why it was promoted (tags). Each prediction records which version made it.
- **Change control.** Promotion is rule-based and logged; rollback is one alias change. In a regulated or safety-critical setting a named approver should sign off promotions as an extra gate.
- **Transparency of limits.** This report states openly where the model is weak (degraded imaging, false-alarm rate) so that operators do not over-trust it.

Limitations

- **Robustness:** defect recall falls to about 0.75 on degraded images and head-only retraining does not restore it.
- **Review load:** roughly one sound part in six is sent for manual inspection.
- **Synthetic drift:** the degraded batch is an artificial transformation of test images, which saturates PSI. Real drift is gradual and would need rolling windows and thresholds calibrated on historical data.
- **Reference set:** monitoring uses clean *test* images as its reference, and the Stage 4.3 cell overwrites the validation-based reference saved by training. One validation-based reference would be cleaner.
- **Evidently** did not run in the final monitoring execution (Section 7); drift decisions rest on PSI.
- **Short training:** validation F1 was still climbing at the fourth and last epoch.
- **Near-duplicates** escape exact hashing.
- **Single dataset and site:** results come from one public collection and one camera set-up; transfer to another foundry is untested.

Recommendations

#	Action	Expected benefit
1	Unfreeze layer4 and fine-tune with a small learning rate on labelled images collected under the degraded conditions	Restore defect recall when imaging changes
2	Extend the release rule: degraded recall $\geq$ champion and clean recall $\geq$ 0.99, in addition to F1	Releases judged on the metric the business cares about
3	Select the decision threshold on validation data subject to recall $\geq$ 0.99	Fewer false alarms at the same safety level
4	Install Evidently in every runtime and keep its HTML report next to PSI	Complete statistical drift evidence
5	Increase the epoch budget while keeping early stopping	Potential further gain in validation F1
6	Harden the container (metrics.json, non-root user, HEALTHCHECK, one requirements file)	Production readiness
7	Replace mean_intensity with dark_ratio; monitor rolling windows of real traffic	Fairer drift share, earlier and readable alerts
8	Add saliency or Grad-CAM views for flagged parts	Inspector trust; detection of shortcut learning

10. Conclusion

The project delivers a working, traceable inspection model. On unseen photographs taken under the dataset's normal conditions it stops 452 of 453 defective impellers, well above the 0.99 recall target, at the cost of sending about one in six sound parts for a second look. Every step that produced this result is reproducible from versioned data and logged runs, and the deployed service reports exactly which model answered each request.

The operational half of the project proved equally important. A simulated deterioration of the imaging set-up was caught by all three monitoring signals, a retraining job was launched automatically, and the resulting candidate was correctly held back because its improvement was too small to trust. The same exercise exposed the model's main weakness – reduced defect recall under poor imaging – and pointed to its cause and remedy: adapting the deeper layers of the network with real examples of those conditions, and judging future releases on recall as well as F1.

References

- [1] R. Dabhi, "Casting product image data for quality inspection," Kaggle dataset, 2020.
<https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product>
- [2] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE CVPR*, 2009, pp. 248–255.
- [4] MLflow Project, "MLflow documentation: Tracking and Model Registry," <https://mlflow.org/docs/latest/>
- [5] FastAPI, "FastAPI documentation," <https://fastapi.tiangolo.com/>
- [6] B. Yurdakul, "Statistical properties of population stability index," Ph.D. dissertation, Western Michigan University, 2018.
- [7] Evidently AI, "Evidently documentation: Data drift," <https://docs.evidentlyai.com/>
- [8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [9] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* 32, 2019.
- [10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems* 28, 2015.

Appendix A. Project files

File	Responsibility
config.py	Paths, label map, hyper-parameters, augmentation, drift and promotion thresholds (provided)
src/data_prep.py	find_data_root, list_images, validate_quality, build_splits, load_split, image_features, get_transforms
src/model.py	build_model, trainable_parameters, parameter_counts, save_model, load_model, EmbeddingExtractor
src/train.py	Seeded weighted training with early stopping, MLflow logging, register_and_promote, reference baseline
src/evaluate.py	Batch prediction, imbalance-aware metrics, confusion matrix and ROC plot, failure-case export
app.py	FastAPI service: /health, /predict, input validation, 400/503 handling, prediction log
src/monitoring.py	Drift simulation, PSI, image-feature drift, Evidently (when installed), embedding drift, confidence
src/retrain.py	Trigger evaluation, drift-aware candidate, registration, promotion/rollback rule, decision record
Dockerfile, .github/workflows/ci.yml	CPU serving image; CI that runs the tests and builds the image

Appendix B. Configuration values

Setting	Value	Setting	Value
IMG_SIZE	224	PSI_THRESHOLD	0.2
BATCH_SIZE	32	DRIFT_SHARE_THRESHOLD	0.4
LEARNING_RATE	0.001	EMBEDDING_DRIFT_THRESHOLD	0.1
WEIGHT_DECAY	0.0001	CONFIDENCE_DROP_THRESHOLD	0.1
EPOCHS / patience	4 / 3	PROMOTE_EPSILON	0.01
VAL_SPLIT	0.15	DRIFT_SIM	brightness 0.6, blur 1.5, noise 12, rotate 8
RANDOM_SEED	42	Retraining	3 epochs, $\leq$ 1,500 images, 40% degraded

Appendix C. Traceability identifiers

Item	Identifier
Production training run	ccb03cd91af0447aaf0a10cdbacd714e
Logged model	models:/m-58cde3d446ee41199f9a3919283d2b37
Live registry version	casting_defect_classifier v4 (@production)
Rejected candidates	v5, v6 (tag rejected_rollback)
Dataset version / fingerprint	v1 / 907a855bb7e9041cb3cfa856a9d0fbd0750dd562ccca959fb41f44c669496f92
CI evidence	GitHub Actions workflow ci, run #1, commit cebcf59 (passed)

Appendix D. Glossary

Term	Meaning in this report
Alias (@production)	Movable MLflow pointer that names the registry version currently served
Embedding	512-value vector produced by the network just before its output layer
False negative / positive	Defective part passed as sound / sound part flagged as defective
Fingerprint	Hash of the whole image inventory, used to detect any change in a dataset version
PSI	Population Stability Index: divergence between two distributions over fixed bins
Recall (defect)	Share of truly defective parts the model flags
Rollback	Keeping or restoring an earlier version as @production instead of a candidate
Stratified split	Division of data that preserves class proportions in each part